

Project Design Phase-II

Implementation & Testing (Client Script & UI Policy)

Field	Value
Team ID	BDU463SWTID-2026-8770
Project Name	Implement Client Script & UI Policy
Date	01 October 2026

Project Objective

Implement Client Script and UI Policy configurations on the ServiceNow Incident form to control field behaviour, validate user input and provide immediate feedback.

Implementation Plan

Step	Activity	Expected Result
1	Identify Incident fields and conditions.	Clear field and condition list.
2	Create UI Policies.	Conditional behaviour is defined.
3	Configure UI Policy Actions.	Field states are applied.
4	Create onLoad Client Script when required.	Initial client-side state is correct.
5	Create onChange Client Script.	Dependent behaviour updates.
6	Create onSubmit Client Script.	Invalid submission is prevented.
7	Use GlideForm methods.	Required client-side operations work.
8	Test all scenarios.	Expected and unexpected cases are identified.

Implementation Principles

- Prefer UI Policy configuration for simple condition-based field states.
- Use Client Scripts for dynamic logic and validation.
- Keep scripts focused on the project requirement.
- Provide understandable validation messages.
- Avoid duplicate logic where possible.

Testing Strategy

Testing verifies that Client Scripts and UI Policies produce the intended Incident form behaviour under different field values and user actions. Every rule should be tested in both applicable and non-applicable conditions.

Test ID	Scenario	Expected Result
TC-01	Open Incident form.	Initial configured field states appear.
TC-02	Change controlling field.	Dependent behaviour updates.
TC-03	UI Policy condition true.	Configured actions are applied.
TC-04	UI Policy condition false.	Non-applicable state is applied.
TC-05	Leave mandatory field empty.	Submission is prevented with guidance.
TC-06	Enter valid values.	Incident can be submitted.
TC-07	Trigger onChange repeatedly.	No unnecessary processing occurs.
TC-08	Read-only condition.	Configured field cannot be edited.
TC-09	Visibility condition.	Field is shown or hidden correctly.
TC-10	Validation message.	Message is clear and relevant.

Test Types

- Functional testing
- Positive-condition testing
- Negative-condition testing
- Form submission validation
- Field-state testing
- Regression testing

Error Handling & Expected Result

Error Handling

- Check field values before dependent logic.
- Use clear user-facing messages.
- Handle empty or unexpected values safely.
- Check for conflicts between UI Policies and Client Scripts.
- Test true and false paths.
- Avoid unnecessary repeated operations.

Validation Checklist

Check	Validation
Client Script events	onLoad, onChange and onSubmit trigger only for intended scenarios.
UI Policy condition	Condition matches the documented requirement.
UI Policy Action	Visibility, mandatory and read-only states are correct.
GlideForm usage	Field operations produce expected results.
Submission	Invalid input is stopped and valid input is accepted.
Messages	Validation messages are understandable.
Regression	Existing Incident behaviour remains unaffected.

Expected Outcome

The completed project provides a controlled Incident form experience. Client Scripts handle dynamic logic and validation, while UI Policies and UI Policy Actions manage condition-based field states.

Completion Criteria

- Required Client Scripts are configured.
- Required UI Policies and Actions are configured.
- Defined Incident scenarios pass testing.
- Documented invalid cases are prevented.
- Final configuration is understandable and maintainable.